CHAPTER 24

Distributed Email Service

Design a Gmail-scale email system: SMTP in/out, storage, search, spam and attachments.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a large-scale email service (Gmail / Outlook): users **send, receive and read email**, search messages, organise with folders/labels, attach files, and are protected from spam and phishing. Support standard protocols (SMTP, IMAP/POP) and a web/mobile API.

Why interviewers ask it: Combines protocols, huge storage with metadata/attachments split, indexing/search, reliability and anti-spam.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Scale?	1 B users.
Features?	Send/receive, folders/labels, search, attachments, spam filter.
Attachments?	Up to 25 MB.
Protocols?	SMTP for transport, HTTP/web for clients, IMAP/POP for legacy.
Retention / durability?	Keep for years; no loss.

3. Requirements

Functional

- Send email (to internal and external domains).
- Receive email and deliver to mailbox.
- Read, delete, organise, mark read/unread.
- Search emails.
- Attachments.
- Spam/virus filtering.

Non-functional

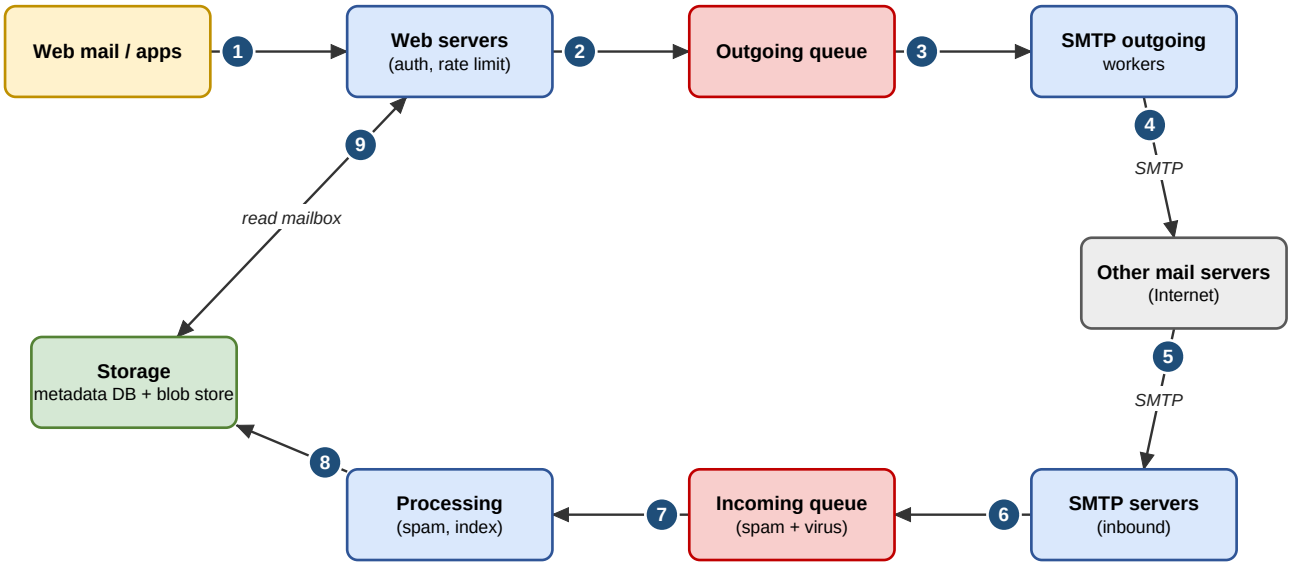
- **Reliability:** never lose email.
- High availability across data centers.
- Scalable storage (exabytes) and throughput.
- Flexible search; fast mailbox open.

4. Back-of-the-envelope estimation

- 1 B users x 10 emails sent/received/day = 10 B emails/day = about **115 K email QPS** (peak 3x).
- Metadata about 50 KB/email (headers + body ~ 100 KB incl. HTML) -> about 1 PB/day text; attachments (10% of emails x 500 KB) = about 0.5 PB/day -> **exabytes over years** -> distributed blob storage + tiering.
- Search index about 10-30% of raw text.

5. High-level design

Sending and receiving



Step	What happens
1	**Send:** the user composes a mail in the web client. `POST /v1/messages` goes to the web servers, which authenticate, rate-limit and validate size and attachments.
2	The message is put on the **outgoing queue**, which gives retries and survives SMTP outages.
3	**SMTP outgoing workers** pick it up, perform DNS MX lookup, sign with DKIM and deliver. Failures are retried with backoff, and bounces are reported back to the user.
4	The worker delivers to the recipient's mail server over **SMTP**.
5	**Receive:** other mail servers deliver to our inbound SMTP servers (behind a load balancer; connection limits and basic validation first).
6	Accepted mail is put on the **incoming queue**, which absorbs spam floods and spikes.
7	Workers run **spam / virus** scanning and checks (SPF, DKIM, DMARC), apply filters and labels, and update the search index.
8	The mail is written to **storage**: metadata (folders, flags, thread) in a wide-column / KV store partitioned by `user_id`, and bodies and attachments in blob storage.
9	**Read:** the user opens the inbox and web servers read metadata (and blobs on demand). Real-time new-mail notification is pushed over WebSocket.

6. Deep dive

Email protocols

- **SMTP** sends/relays mail between servers (port 25/587). **POP3** downloads and typically removes; **IMAP** keeps mail on the server, syncs folders and flags; web uses **HTTP/REST** (Gmail). DNS **MX records** locate a domain's mail servers.

Sending flow

- Client -> web server (validate, size, rate limit) -> put on the **outgoing queue** (so retries do not block the user) -> SMTP workers look up the recipient's **MX record** and deliver; on failure retry with backoff, send bounce notification when final.
- Outgoing queue stats help detect delays and misbehaving senders.

Receiving flow

- Inbound SMTP servers accept mail -> **incoming queue** -> anti-spam/anti-virus -> parse/split metadata and attachments -> store in mailbox storage -> push to online clients (WebSocket) or let them poll.
- Queues absorb floods (spam waves, mass mailings) so the mailbox storage is not overwhelmed.

Storage design

- **Metadata** (sender, recipients, subject, folder, flags, thread id, timestamps, snippets) in a scalable DB keyed by `(user_id, folder, timestamp)` (Bigtable/Cassandra-style, partitioned by user). Query pattern: list recent emails of a folder; update flags; threads. Strong consistency per user mailbox is preferred.
- **Bodies and attachments:** blob/object storage (HDFS/S3-like) referenced by id; attachments deduplicated by hash (one large attachment sent to many users stored once).
- Hot data (recent 30 days) on SSD/cache; old mail to cold storage (cheaper). Compress.

Search

- Email search is per-user and write-heavy: new mail continuously indexed. Use **Elasticsearch** (optionally sharded by user) for full-text; or a custom inverted index in the same datastore (LSM) to avoid inconsistency/double-write. Indexing is async from the incoming pipeline.
- Trade-off: ES is flexible but needs sync with primary storage; a custom index couples tightly with storage and scales per user.

Spam and security

- Spam filter: sender reputation, SPF/DKIM/DMARC checks, ML content classifiers, user feedback, attachments scanning in sandbox; rate limiting. Encryption in transit (TLS) and at rest.

Deliverability and reliability

- Replicate mailbox data across data centers; multi-DC active-active with per-user home DC; queue durable storage; idempotent delivery (message-id dedupe).

Consistency and failover

- Users read their own writes: route a user's requests to the home DC; if it fails, fail over to a replica with eventual catch-up.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Metadata store	SQL / wide-column NoSQL	Wide-column (Bigtable/Cassandra) for scale, partitioned by user.
Search	Elasticsearch / custom index in store	ES to start; custom for extreme scale and tight consistency.
Attachments	Inline in DB / blob store + dedupe	Blob store with dedupe.
Client sync	Polling / push (WebSocket)	Push for freshness, with polling fallback.

8. Scaling, reliability, failure modes

- Partition by user_id everywhere; add shards linearly.
- Queues decouple SMTP front ends from storage.
- Tiered storage and compression keep the cost curve manageable.

9. Likely follow-up questions

- How do you make email delivery reliable when the recipient server is down?
- How do you detect and block spam at scale?
- How would you implement threading/conversations and labels?
- How do you handle legal hold, deletion and data residency?

10. Common mistakes

- Storing attachments in the metadata DB.
- Synchronous send (user waits for remote SMTP).

- Ignoring spam/security.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Scope: send, receive, folders and labels, search, attachments, and spam protection, for 1 B users. Assume 10 emails per user per day, that is about 100 K sends per second, with 50 KB average size and 10% attachments. Storage is in the exabyte range over years, so tiering and dedup matter. Availability and no mail loss come first."
2. High-level design	10 min	"Outbound: web servers, an outgoing queue, SMTP workers. Inbound: load-balanced SMTP servers, incoming queue, spam and virus processing, then storage. Storage splits into a metadata DB partitioned by user, and a blob store for bodies and attachments. A WebSocket pushes new mail."
3. Deep dive	25 min	"Metadata needs fast per-user reads, so I use a wide-column store keyed by user and folder. Bodies and attachments go to object storage, and I dedupe attachments by hash. Search is a per-user inverted index. For reliability, queues give retries and backoff, and I replicate across data centres. Spam protection includes SPF, DKIM, DMARC, reputation and ML. I'd explain MX lookup and the SMTP / IMAP / POP roles."
4. Wrap up	4 min	"Extras: cold storage for old mail, thread view, per-domain rate limits and monitoring queue depth and bounce rate. For disaster recovery, replicate mailboxes to a second region."

12. 60-second recap / practice checklist

- SMTP in/out with queues; mailbox = metadata store (by user) + blob store (attachments, dedupe).
- Async indexing for search; spam/virus pipeline before delivery.
- Multi-DC replication, tiered storage.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• RFC 5321: SMTP
• RFC 9051: IMAP4rev2
• Google: Email sender guidelines (SPF, DKIM, DMARC)